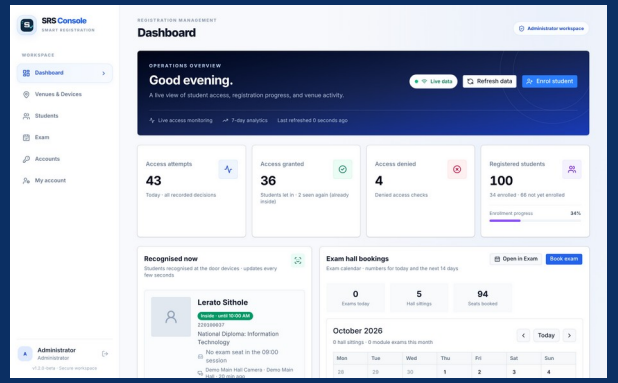
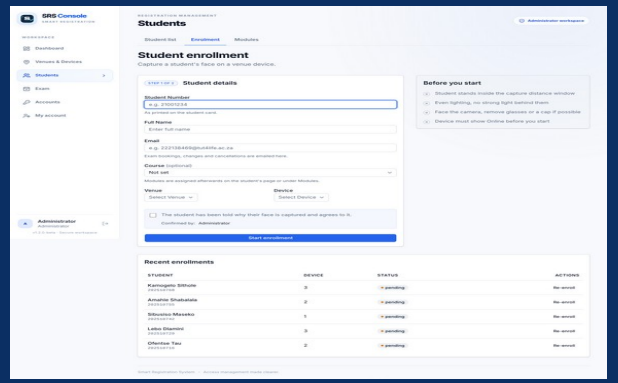
Tshwane University of Technology

We empower people

Smart Registration System

Face-recognition access control for university exam venues





TC Mance · 222138469 · Project Design 376 (PD2316D)
Lecturers: Mr M. Muwanguzi Ntume and Prof C. Du · 5 November 2026

Checking identity by eye is slow and easy to fool

Slow at the door

Every candidate is compared with a register and a student card

Interrupts writing

Checks are repeated during the exam, disturbing students

Impersonation slips through

A convincing look-alike passes a quick visual check

No reliable record

Paper ticks give no time-stamped audit trail

The aim

Verify every candidate automatically at the venue door

Show each student's seat and session at the door

Keep the invigilator in charge of every refusal

Four objectives from the proposal – all met

Fast, distance-gated recognition

MET

Goal: Face verified within 2 s at 90%+ accuracy

≈1 s hold at 25–50 cm, then the face is matched

Low FAR and FRR

MET

Goal: Keep both error rates acceptable

Strict 0.40 match threshold · one face per student

Log every attempt, live dashboard

MET

Goal: Time, student and decision for each entry

Every decision stored; live console for all venues

Manual override

MET

Goal: Backup when recognition fails

Override button records who and why

Face recognition exists – exam-aware access does not

Commercial products

Facephi · SmartVisitor · Navon · Openitem

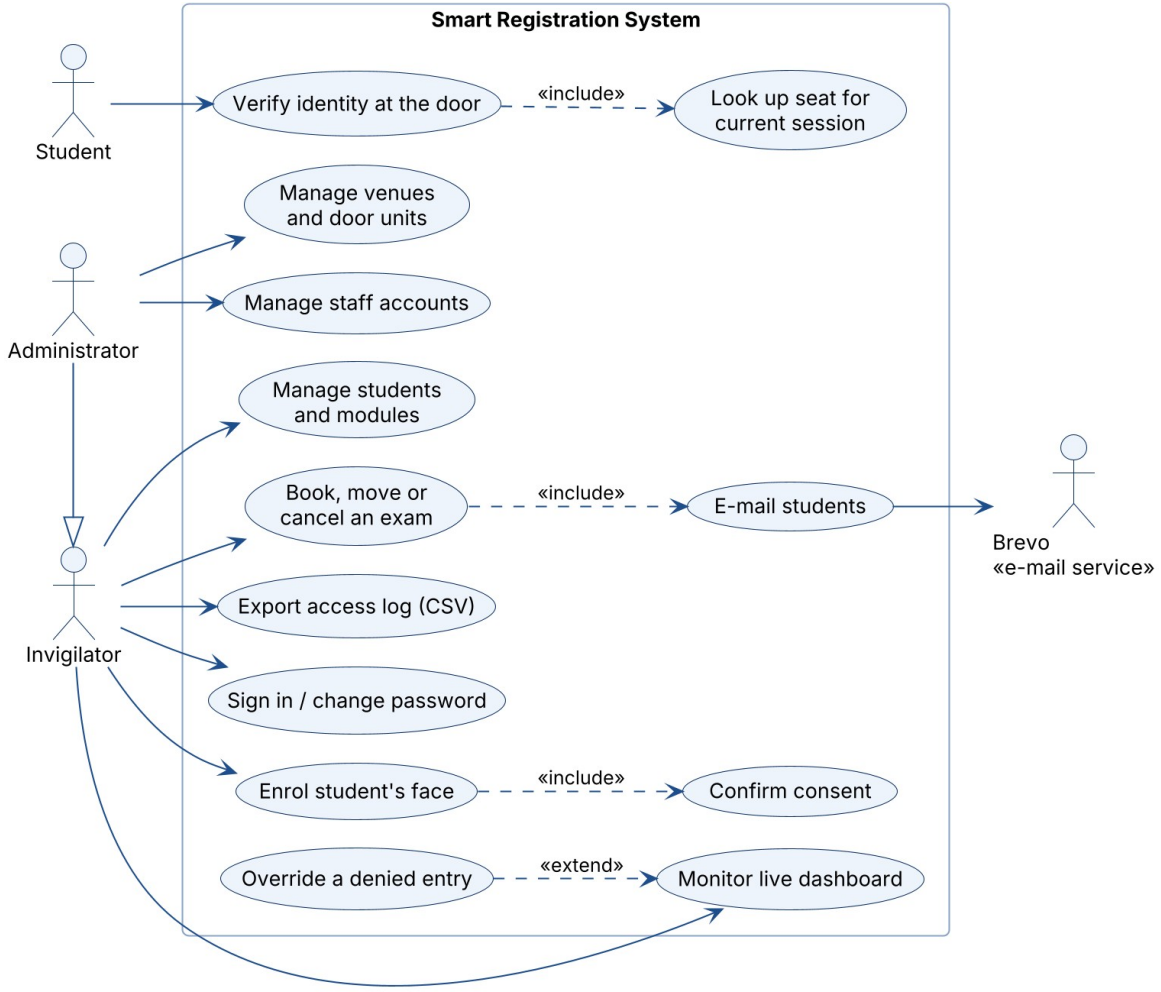
- Offices, estates, events and visitors
- No idea of exam timetables or seats
- Face templates usually kept in the cloud
- Subscription – about R480 per month

Smart Registration System

Built for TUT exam venues

- **Shows each student's seat; flags "No seat" for the invigilator**
- **Once per session; automatic sign-out at 12:00 / 16:00**
- **Face templates stay on the door PC (POPIA)**
- **About R1 225 of hardware, free hosting tiers**

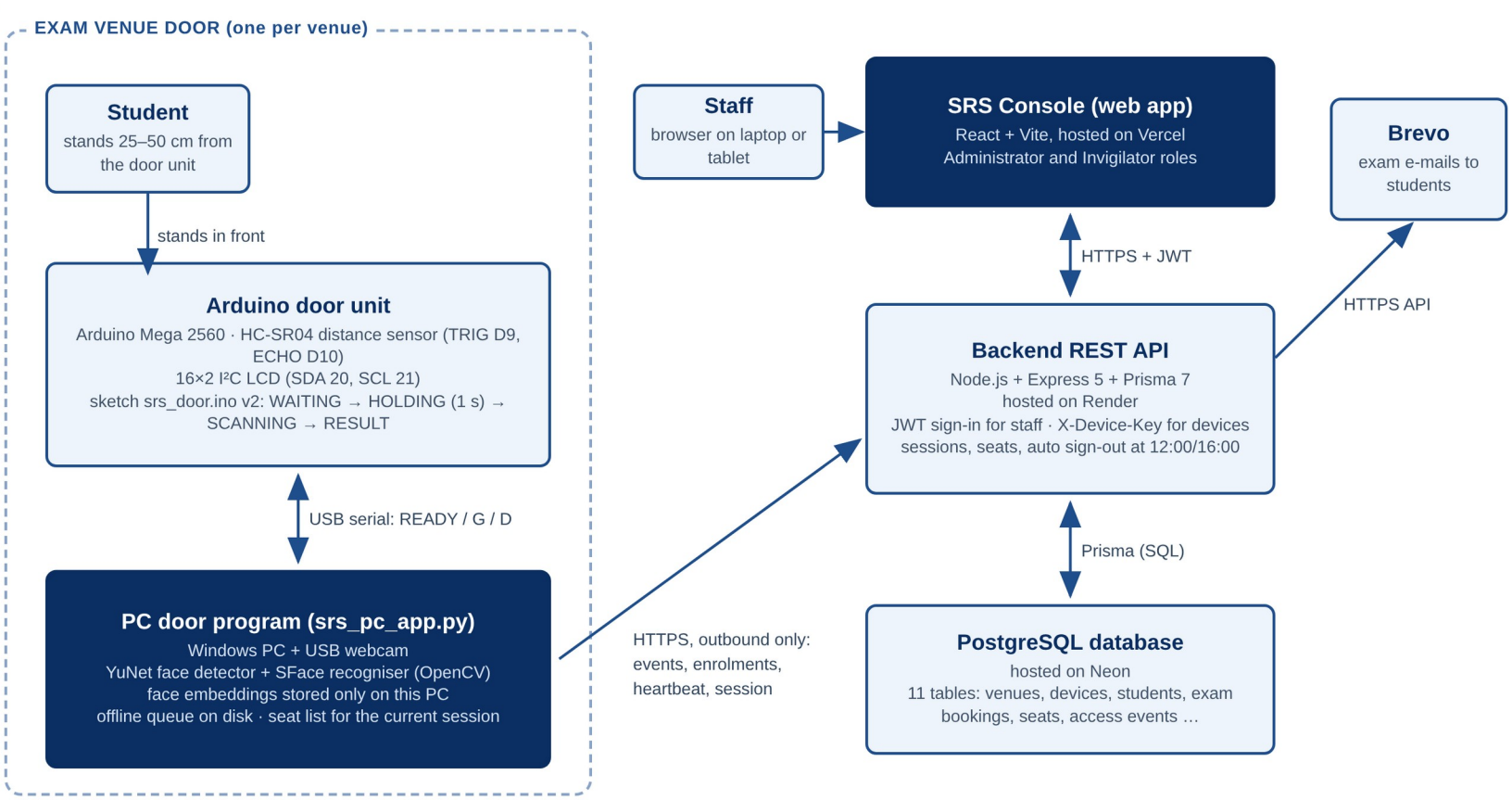
Who uses the system: UML use case diagram



Key functions

- **Enrol a face:** registers a student's face after consent
- **Students and exams:** manage students and modules; book, move or cancel exams (students are e-mailed)
- **Monitor:** live dashboard, overrides, CSV export of the log
- **Verify at the door:** recognise the student and look up their seat
- **Administer:** venues, door units and staff accounts (admins only)

Four parts, connected only through the backend



Door unit

PC + webcam recognises; Arduino measures distance, shows the result

Backend

Node.js + Express on Render; decides sessions, seats, sign-out

Security

Staff sign in (JWT); doors use a secret device key; outbound HTTPS only

The hardware change: ESP32-CAM to PC + Arduino

Planned

- ESP32-CAM with OV2640 camera
- ESP-WHO recognition on the board
- Battery pack + buck converter
- Status LED for feedback

Final

- **USB webcam on a Windows PC**
- **OpenCV YuNet + SFace recognition**
- **USB power – no batteries**
- **Arduino Mega: distance sensor and LCD**

Why: hardware problems with the ESP32-CAM could not be solved in time. The new design removed its memory, power and pin limits – and the new door unit uses the same backend API.

The door unit waits for the student to hold still



25–50 cm

distance window (HC-SR04)

1 s

hold before the picture

G / D / A

answer shown on the LCD

d = 343 m/s × t ÷ 2 → 25–50 cm = 1.45–2.9 ms echo

From face to decision: match first, then session rules

1. Hold

Student holds still 1 s at 25–50 cm

2. Detect

YuNet finds the largest face

3. Match

SFace: 128 numbers, compared with ALL faces, ≥ 0.40

4. Session

Already inside? Session over? Seat looked up

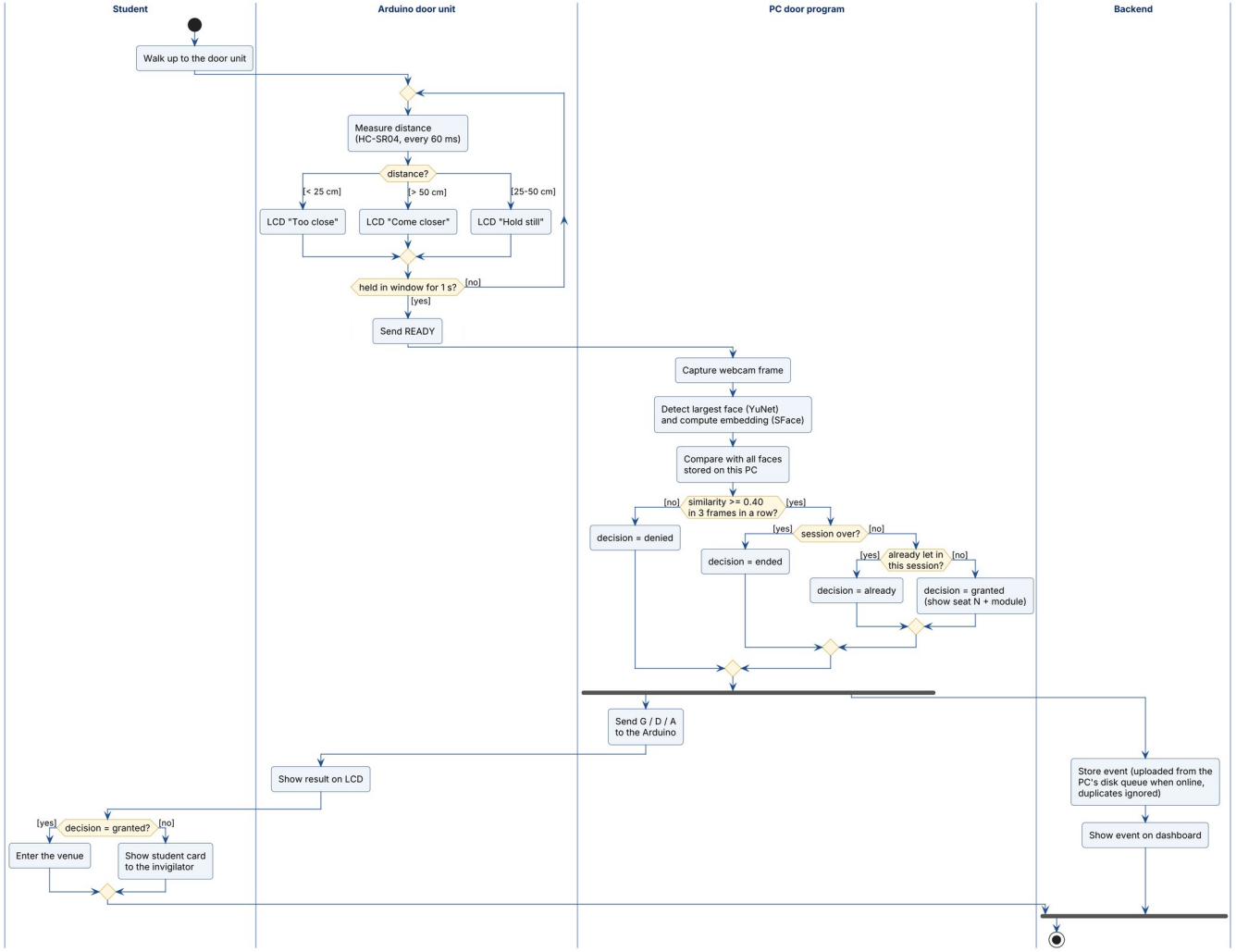
5. Decide

Let in with seat N (or "no seat" flag); unknown face DENIED

Offline? Decisions continue; events are queued on disk and uploaded once, never twice

Face templates stay on the door PC; the backend only gets the result and one enrolment photo

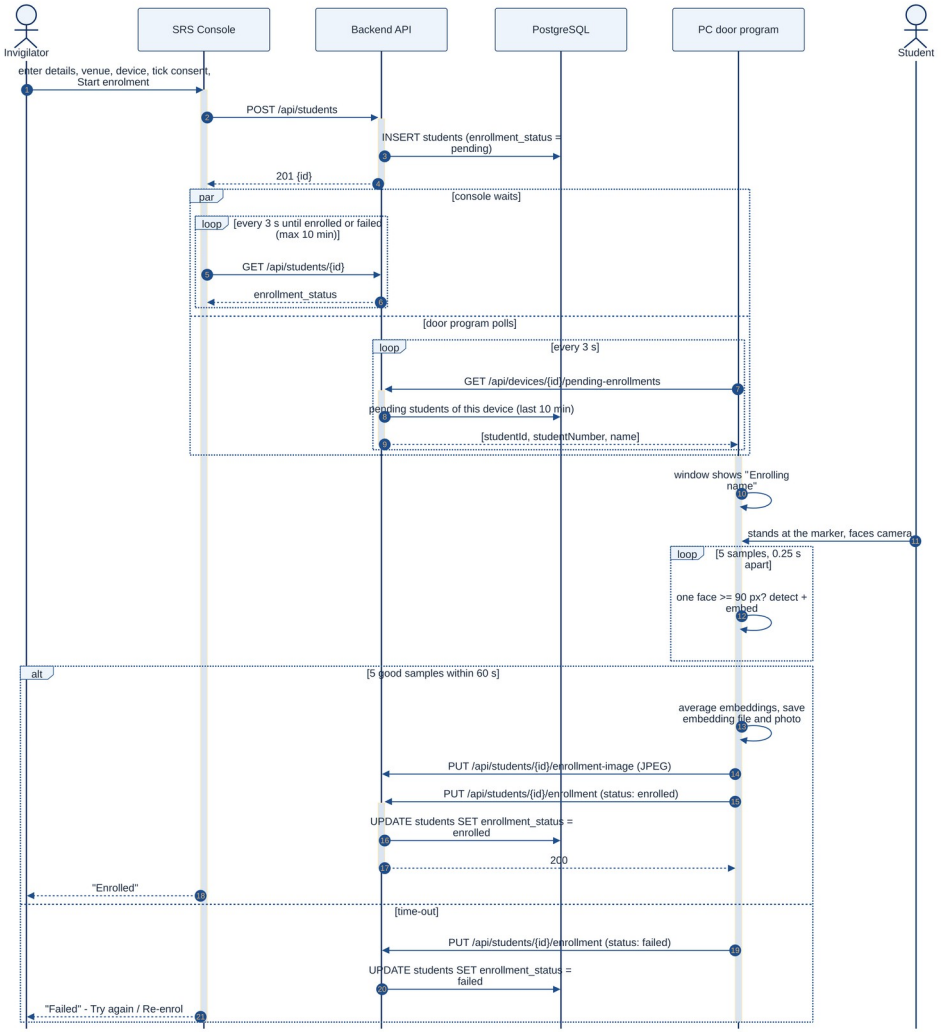
Access verification: UML activity diagram



Key functions

- **Distance gating:** 25–50 cm, measured every 60 ms, with LCD prompts
- **Hold to scan:** after 1 s in the window the Arduino sends READY
- **Face matching:** YuNet + SFace; similarity ≥ 0.40 in 3 frames in a row, otherwise denied
- **Session rules:** session ended, already inside, or granted with a seat
- **Result and log:** G / D / A on the LCD; event stored and shown on the dashboard

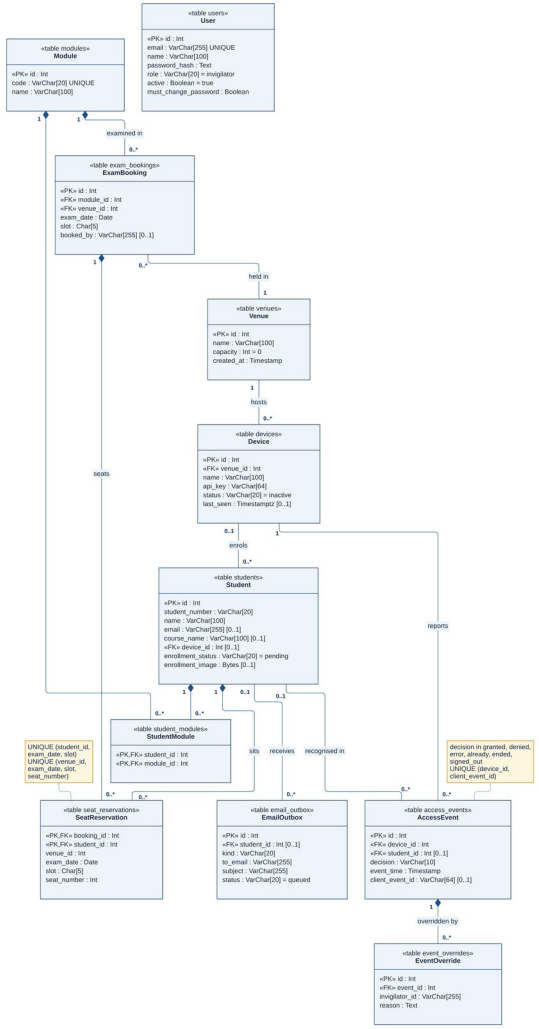
Enrolment: UML sequence diagram



Key functions

- **Start enrolment:** details and consent saved; the student is pending
- **Request polling:** console and door check the backend every 3 s (10 min limit)
- **Face capture:** 5 samples, 0.25 s apart, face at least 90 px wide
- **Save and report:** embedding kept on the PC, photo uploaded, status enrolled
- **Time-out:** no 5 good samples in 60 s, so the status is failed; try again

Database: UML class diagram (11 tables)



Key functions

- **Venues and door units:** each venue hosts its door units
- **Enrolment:** each student is enrolled on one door unit, with status and photo
- **Access log:** every decision is an access event; overrides record who and why
- **Exam seating:** bookings place a module in a venue; one seat per student per session
- **Staff and e-mails:** staff accounts with roles; exam e-mails queued per student

SRS Console: one app for invigilators and admins

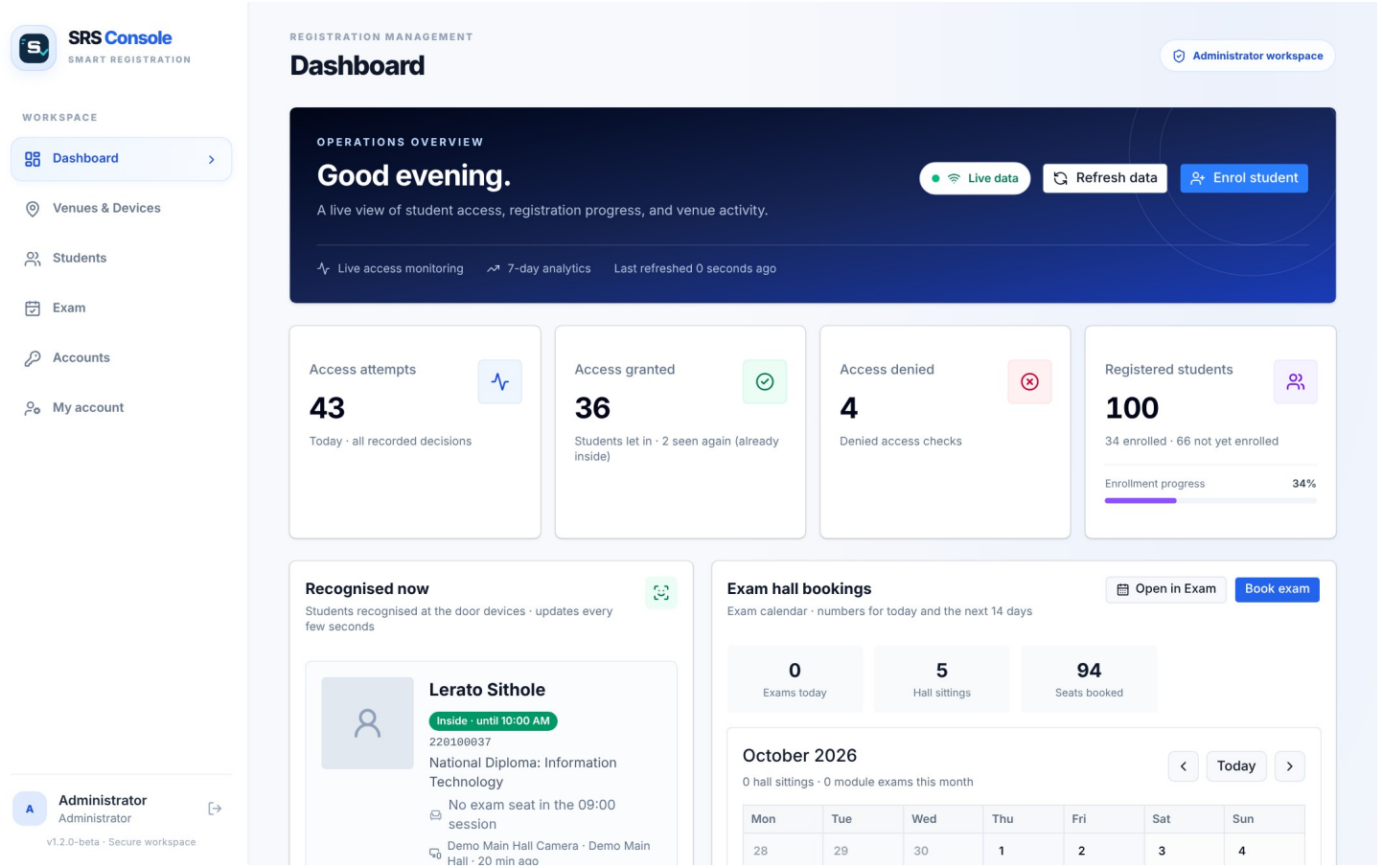
Live dashboard
entries, refusals, recognised now

Enrolment
with consent tick box

Exam bookings
seats, capacity, calendar

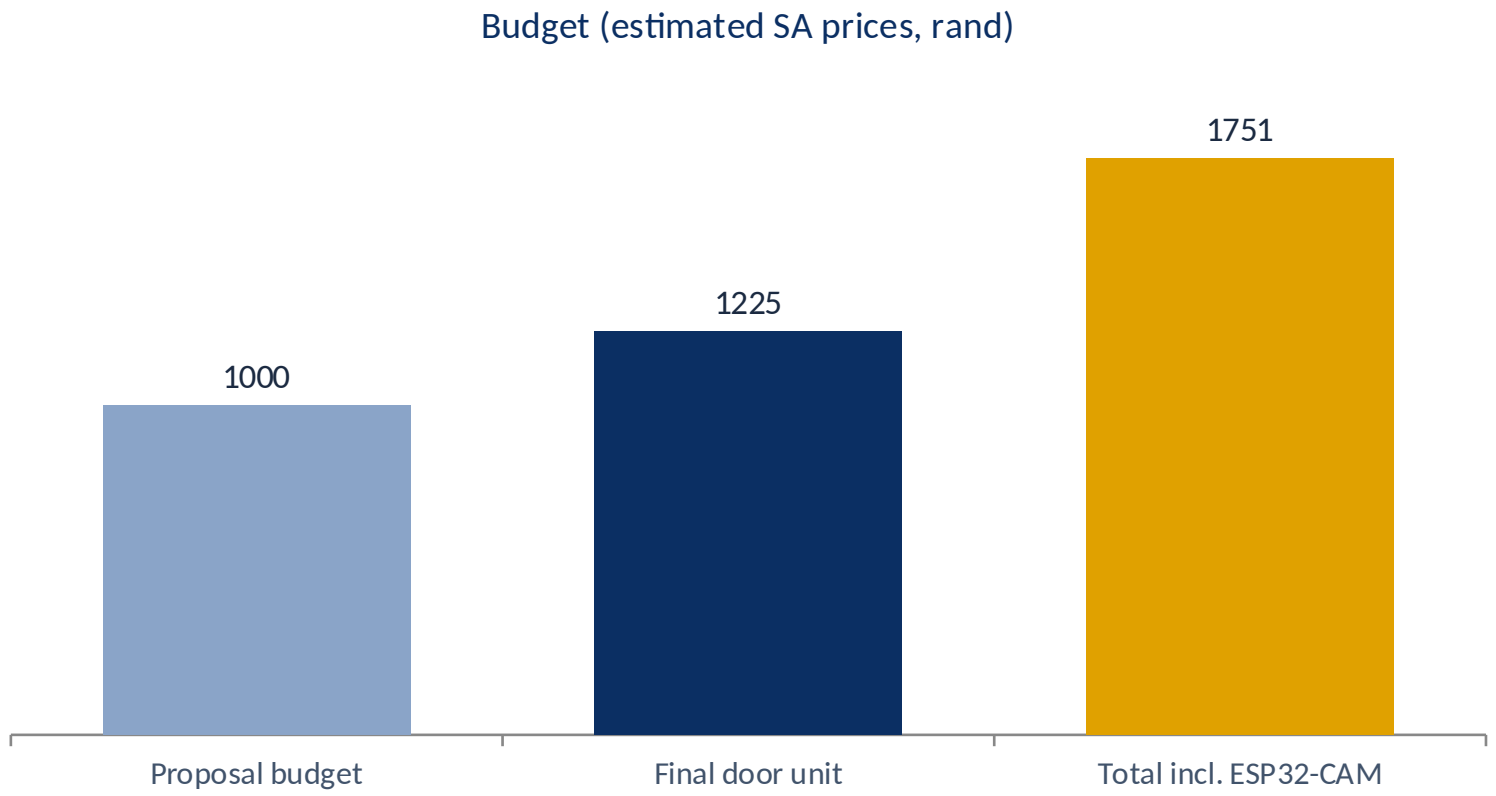
E-mails
booked, moved, cancelled

Two roles
admin and invigilator



Demo data shown

Costed in rand



Over budget because two sets of hardware were bought after the ESP32-CAM was dropped.

Conclusion and next steps

What was achieved

- All four proposal objectives met
- Recognition tied to exam bookings and seats
- Biometric templates stay on the door unit
- Grew to many venues, e-mails and staff roles

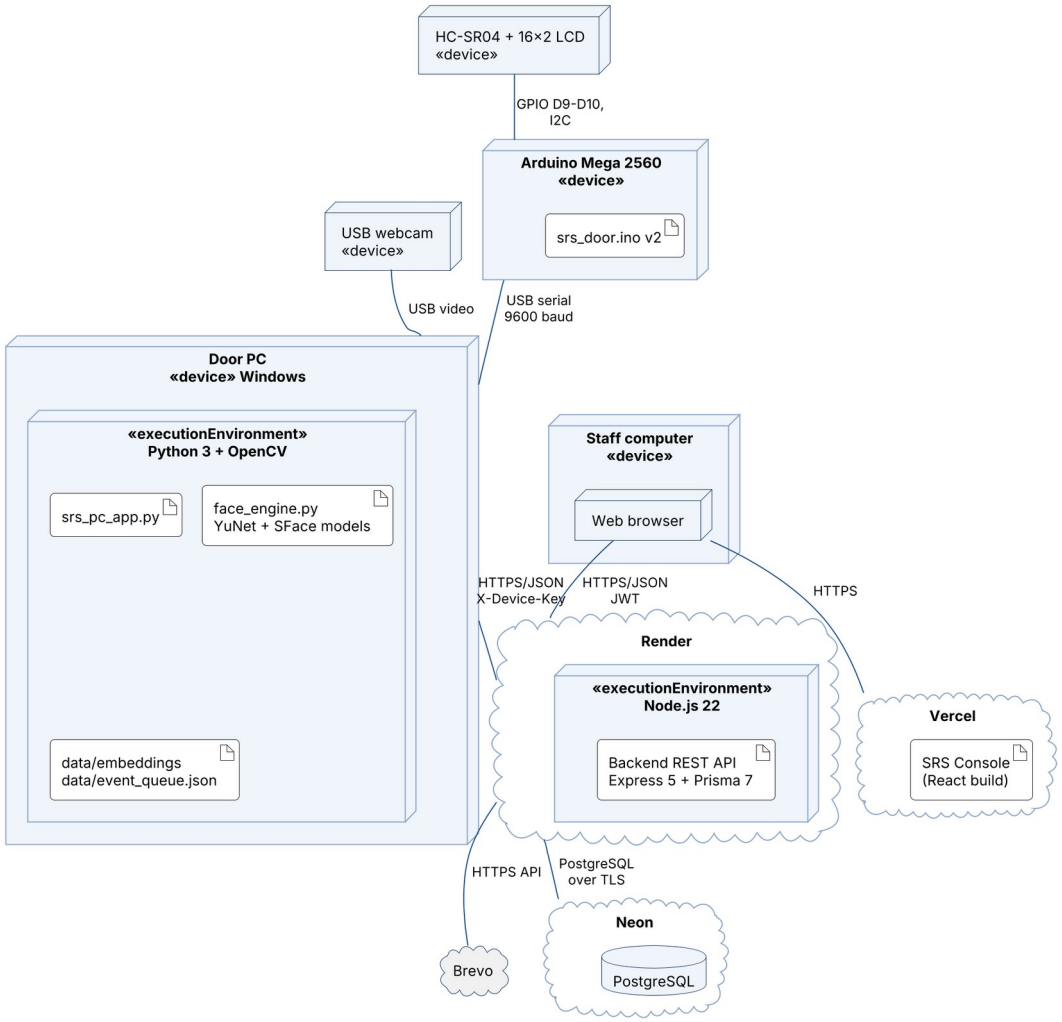
Next steps

- On-site FAR/FRR test with 30+ volunteers
- Liveness check against printed photos
- Move the door program to a Raspberry Pi 5
- Import students from the university system

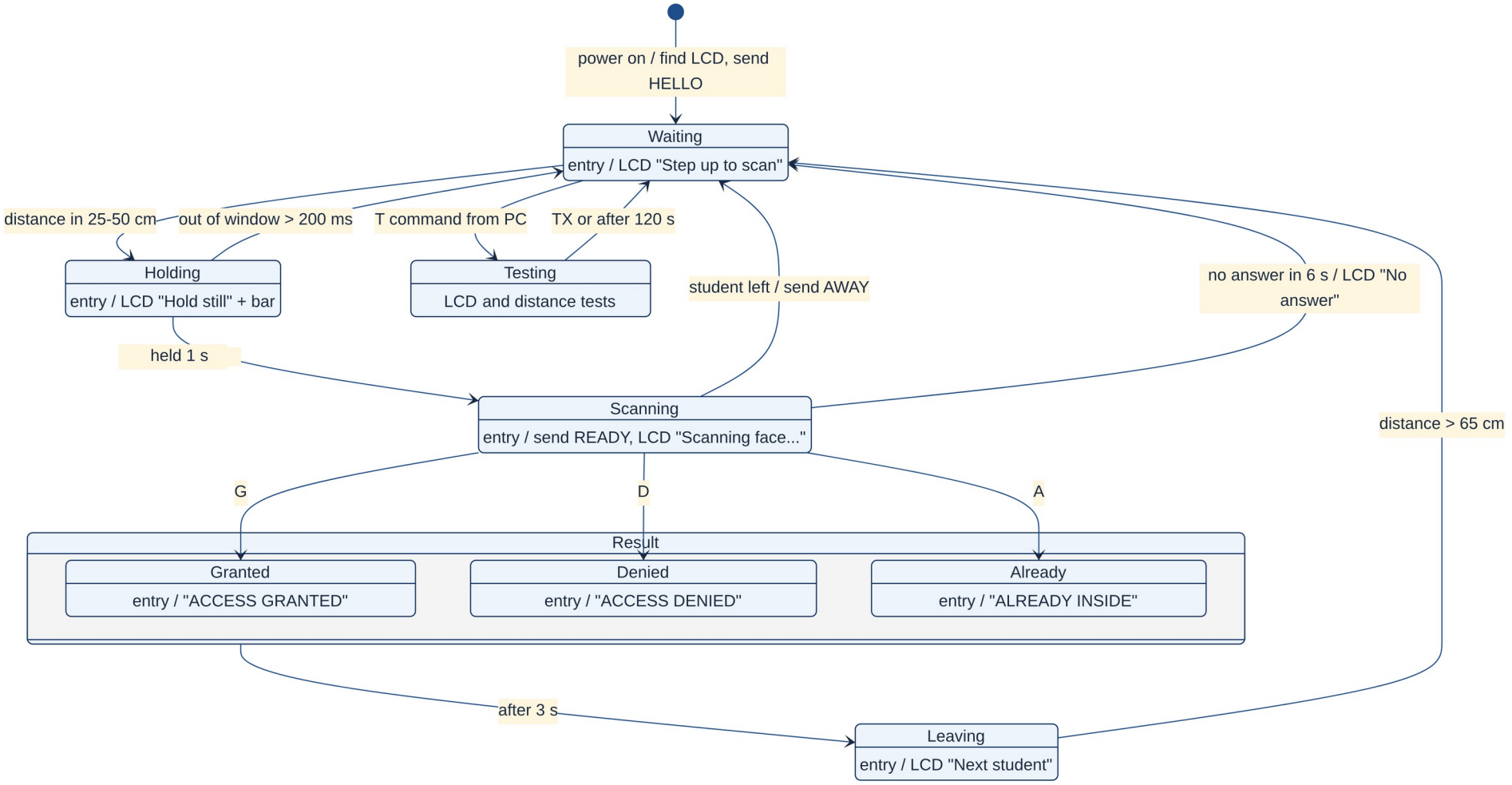


Thank you – live demonstration and questions

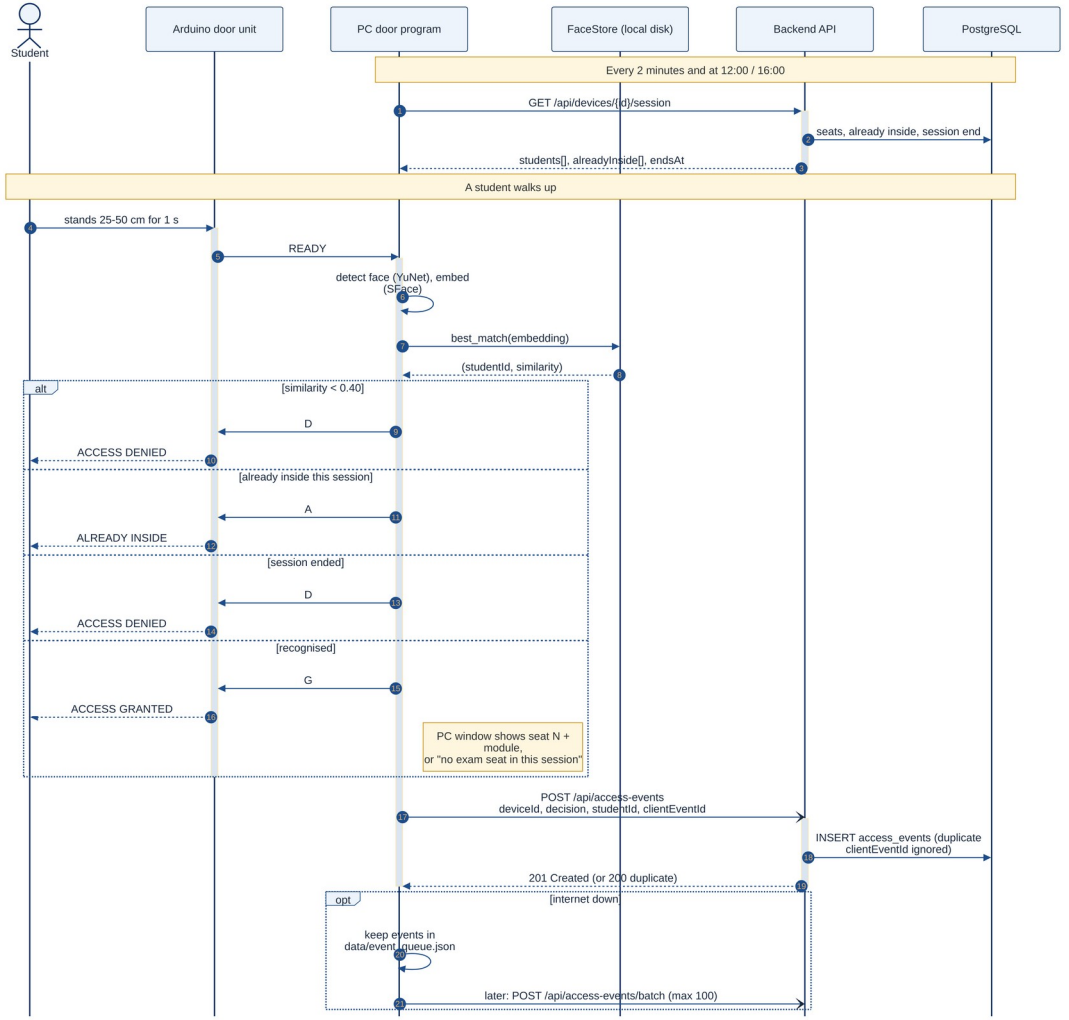
Backup: UML deployment diagram



Backup: UML state machine of the door unit



Backup: UML sequence – one access verification



Backup: UML sequence – booking, e-mails, sign-out

